

Lab 11: Network Security & The Control Plane

HTTP, HTTPS, and the TLS Handshake

Bernal Manzanilla

Concordia University of Edmonton
IT 213: Networking

Winter 2026

How to Use This Presentation

- **Purpose:** This slide deck bridges the theoretical security concepts from Kurose & Ross Chapter 8 with the practical command-line tools we use in the Linux terminal.
- **Structure:** We will define the core tenets of network security, explain how the Transport Layer Security (TLS) handshake works, and then break down the exact Linux commands we use in the lab to capture this traffic.
- **Textbook Alignment:** Specific Kurose & Ross Chapter 8 sections are referenced for further reading.

What is Network Security? I

Reference: Kurose & Ross, Section 8.1

Standard network protocols (IP, TCP, UDP) were designed for efficiency and delivery, not for security. They do not inherently protect your data. If someone intercepts the packet, they can read the payload. Secure communication requires three main principles:

- ❶ **Confidentiality:** Only the sender and the intended receiver should be able to understand the contents of the transmitted message. This is achieved through encryption.
- ❷ **Message Integrity:** The receiver must be able to verify that the message was not altered in transit (either maliciously or by accident).
- ❸ **End-Point Authentication:** The sender and receiver must be able to confirm the true identity of the other party involved in the communication.

The Vulnerability of HTTP (Port 80) I

Reference: Kurose & Ross, Section 8.6

The Hypertext Transfer Protocol (HTTP) is the foundation of the web, but in its native form, it is highly insecure.

- **Cleartext Transmission:** HTTP transmits all data—including passwords, emails, and session cookies—in plain, human-readable text.
- **The Threat Model:** Because wireless networks (like coffee shop Wi-Fi) broadcast physical signals in all directions, any node with a network adapter in "promiscuous mode" can run a packet sniffer (like Wireshark) and read everything you do.

The Vulnerability of HTTP (Port 80) II

What an intercepted HTTP packet looks like in Wireshark:

```
GET /login.php HTTP/1.1  
Host: insecure-bank.com  
Cookie: session_id=12345ABC
```

```
POST Payload: username=admin&password=secretpassword123
```

As you can see, there is zero confidentiality. The attacker now has the user's password and session cookie.

The Solution: HTTPS and TLS I

Reference: Kurose & Ross, Section 8.6 (Securing TCP)

To secure web traffic, we use **HTTPS** (HTTP Secure) over Port 443.

- **The Architecture:** HTTPS is not a completely new protocol. It is simply standard HTTP traffic wrapped inside a cryptographic protocol called **Transport Layer Security (TLS)**.
- **The Mechanism:** TLS sits directly on top of TCP. The application layer hands plaintext data to TLS, TLS encrypts it, and hands the ciphertext down to TCP for transmission.
- **The Result:** If an attacker intercepts an HTTPS packet, they will only see the IP headers and TCP headers (so they know **who** you are talking to), but the payload (the **what**) will appear as randomized, scrambled garbage.

The TLS Handshake: Step-by-Step I

Reference: Kurose & Ross, Section 8.6.2

Before a client (your browser) and a server (Google) can send encrypted data, they must agree on how to encrypt it. This negotiation is called the **TLS Handshake**.

Step 1: Client Hello

The client initiates the connection. It sends a message saying, "I want to connect securely. Here is the list of cryptographic algorithms (cipher suites) that I support."

Step 2: Server Hello

The server looks at the client's list and replies, "I also support those. Let's use this specific algorithm."

The TLS Handshake: Step-by-Step II

Step 3: The Digital Certificate (Authentication)

(Reference: Kurose & Ross, Section 8.3.3)

How do you know you are actually talking to Google and not an attacker intercepting your connection? The server sends an **X.509 Digital Certificate**. This document is digitally signed by a trusted third-party Certificate Authority (CA) and binds the server's public key to its true identity.

Step 4: Key Exchange

Using complex public-key cryptography (like RSA or Diffie-Hellman), the client and server securely generate a shared, secret "Symmetric Key."

Step 5: Secure Communication

The handshake is complete. Both sides now use that shared secret key to rapidly encrypt and decrypt all HTTP "Application Data" for the rest of the session.

Automating the Capture: tcpdump I

In the lab, we use a bash script to capture traffic. Let's break down the exact `tcpdump` command we are executing line-by-line.

```
tcpdump -i any tcp port 80 or tcp port 443 -n -U -w trace.pcap &
```

- `tcpdump`: The command-line packet analyzer (the terminal version of Wireshark).
- `-i any`: Instructs the program to listen on all available network interfaces.
- `tcp port 80 or tcp port 443`: The filter. It tells `tcpdump` to ignore background noise (like ARP or DNS) and only capture unencrypted web traffic (80) and encrypted web traffic (443).

Automating the Capture: tcpdump II

- `-n`: Prevents tcpdump from trying to resolve IP addresses into hostnames. This makes the capture significantly faster.
- `-U`: Forces packet-buffered output. This is a critical safety flag in scripts; it ensures that every packet is immediately written to the disk so the `.pcap` file isn't corrupted when the script suddenly kills the process.
- `-w trace.pcap`: Instructs the program to **w**rite the raw, binary packet data to a file named `trace.pcap` so we can download it and open it in Wireshark.
- `&`: Executes the packet sniffer in the background, allowing the bash script to continue to the next line to generate the web traffic.

Generating Traffic: curl I

While `tcpdump` is sniffing in the background, our script uses the `curl` command to act as a command-line web browser and generate traffic.

Generating Insecure Traffic:

```
curl -s http://neverssl.com > /dev/null
```

- We request a website using `http://`. We specifically use `neverssl.com` because modern websites (like Google) will forcefully redirect you to HTTPS. This guarantees we generate pure port 80 cleartext traffic.
- The `> /dev/null` portion throws away the downloaded HTML so it doesn't clutter our terminal screen.

Generating Traffic: curl II

Generating Secure Traffic and Logging the Handshake:

```
curl -v https://www.google.com > tls_handshake.txt 2>&1
```

- **-v (Verbose):** This is the most important flag. It forces curl to print the entire TLS Handshake (the Client Hello, Server Hello, and Certificate exchange) to the terminal output.
- **https://:** Forces the connection over secure port 443.
- **2>&1:** A standard Linux redirection trick. The verbose handshake output is technically written to the "Standard Error" stream. This operator catches it and flawlessly redirects it into our `tls_handshake.txt` file so you can read it for your lab report.